

Git & GitHub

Version control, repositories, commits, branches, collaboration and a basic automation workflow.

Git and GitHub are different

Git is a distributed version-control system. GitHub is a platform that hosts Git repositories and adds collaboration, Pull Requests, Issues, Actions and other services.

Installation and first configuration

Install Git from the official site and validate with `git --version`. Configure author name and email with `git config`. In corporate environments, follow identity and authentication policies.

Basic workflow

`git status` shows state; `git add` stages changes; `git commit` records a version; `git push` sends commits; `git pull` integrates remote changes; `git clone` gets a repository for the first time.

Branches

Branches allow isolated development. `git switch -c` creates a branch; `merge` combines histories. Teams may use trunk-based development or other branching strategies.

Pull Request and Code Review

A Pull Request proposes integration and creates space for review, tests and discussion. QA can review impact, added tests, risks and evidence.

Conflicts

Conflicts occur when Git cannot automatically decide how to combine changes. Resolve content consciously, mark it resolved and test before committing.

Automation good practices

Do not version `.venv`, secrets, tokens, temporary evidence or caches. Use `.gitignore`, small commits and messages that explain the change.

Git in the QA lifecycle

Versioning automation, synthetic data, documentation and pipelines provides traceability. History helps explain when and why a test changed.

QA tip

Before `git push`, use `git status` and review what will actually be sent.

Official resources

Site oficial: <https://git-scm.com/downloads>